EWB

EWB CLOUD LABS | STUDENT PROJECT REPORT

COURSES

BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.

Prove private connectivity.

Project report: EC2, custom route tables, security groups and a bastion host.

VPC A 10.10.0.0/16 <-- pcx --> VPC B 10.20.0.0/16

2 VPCs / 3 EC2 INSTANCES / 1 PRIVATE CONNECTION

AWS ARCHITECT ASSOCIATE

Name	Dnyanesh Dattatray Dambe
Batch	AWS Batch 1
Project title	VPC Peering with Bastion Host (Project 01)
AWS Region	Asia Pacific (Sydney) ap-southeast-2
Completion date	10 November 2026

LEARN. ELEVATE. EMPOWER.

01 / PROJECT BRIEF

Project overview

What I built and what this report proves.

This project connects two separate AWS VPCs with a VPC peering connection so that two private EC2 servers can exchange traffic using only their private IPv4 addresses. A single public bastion host in VPC A is the only entry point from the internet, and I used it to administer both private servers over SSH with ProxyJump.

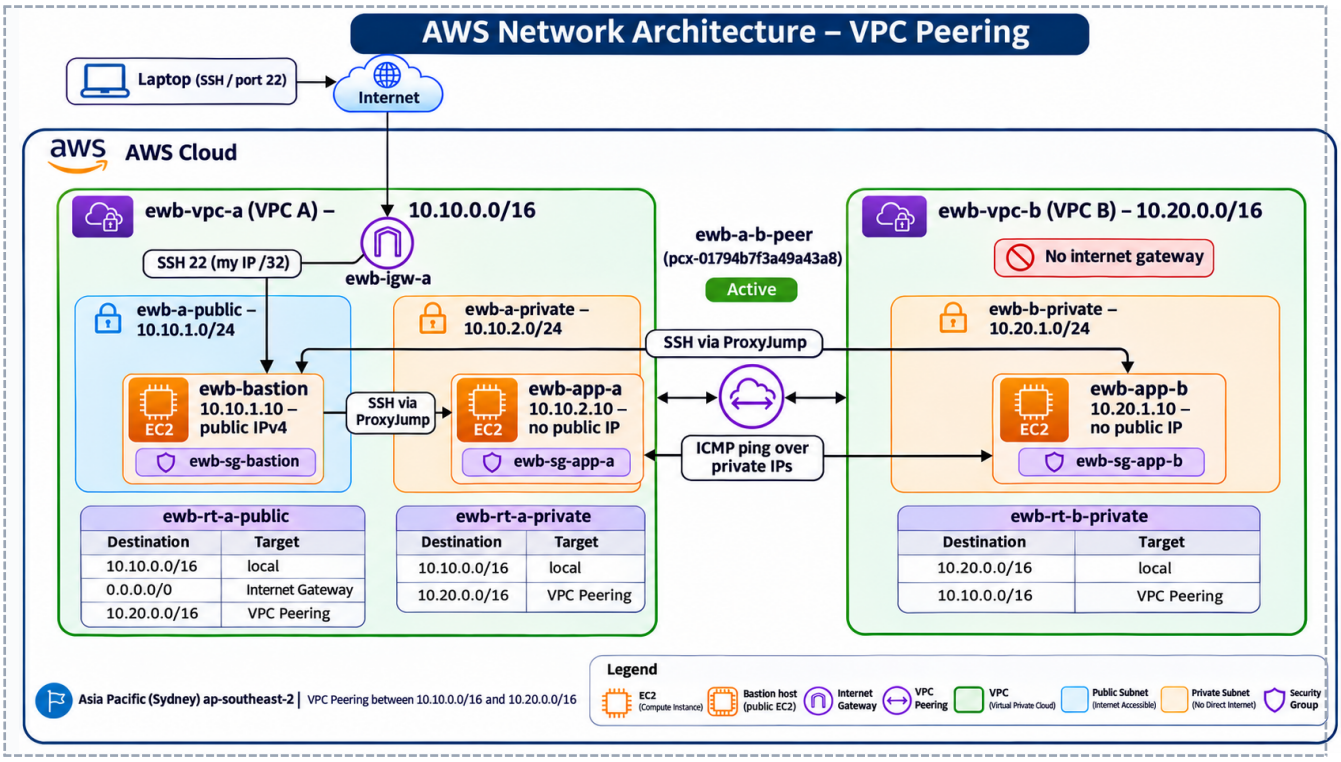
What this report demonstrates

- Non-overlapping VPC and subnet CIDR planning (10.10.0.0/16 and 10.20.0.0/16).
- Public and private subnets with explicit route table associations.
- Peering with routes on both sides and tightly scoped /32 security group rules.
- A failed test before peering, successful tests after peering, and a controlled failure with recovery.

02 / ARCHITECTURE

The network I built

One internet entry point. Two private workloads. A direct peering connection between their VPCs.



Administration path: laptop > internet gateway > bastion > App-A or App-B using SSH ProxyJump. Project traffic: App-A <> App-B over private IPs and the peering connection. The bastion opens a separate SSH connection to the destination. Peering does not give VPC B internet access through VPC A, and VPC B has no internet gateway.

03 / RESOURCE PLAN

Address plan and inventory

Names, CIDRs and the actual AWS IDs from my build.

Resource / Name tag	VPC or subnet	CIDR / address	Actual AWS ID
ewb-vpc-a	VPC A	10.10.0.0/16	vpc-07c4dfaa1590820dc
ewb-a-public	VPC A / public	10.10.1.0/24	subnet-065478ade9e49048b
ewb-a-private	VPC A / private	10.10.2.0/24	subnet-0b173ff9f653ac192
ewb-vpc-b	VPC B	10.20.0.0/16	vpc-098d11add1666ab2f
ewb-b-private	VPC B / private	10.20.1.0/24	subnet-01448ae121e400e3c
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4	i-0ec2a3868c42a86a1 public IP 3.25.110.70
ewb-app-a	ewb-a-private	10.10.2.10, no public IP	i-035129c937086c3fe
ewb-app-b	ewb-b-private	10.20.1.10, no public IP	i-038649aca2929db3c
ewb-igw-a	Attached to VPC A	-	igw-02424954962a7586e
ewb-rt-a-public	VPC A	-	rtb-09a03492fb26cbda6
ewb-rt-a-private	VPC A	-	rtb-0d033ec3d570c2627
ewb-rt-b-private	VPC B	-	rtb-0fbc8e8a4e4e5e4b2
ewb-sg-bastion	VPC A	-	sg-077c8face12c47b34
ewb-sg-app-a	VPC A	-	sg-09d54e87a4f89f12f
ewb-sg-app-b	VPC B	-	sg-0656046045b728c5f
ewb-a-b-peer	VPC A <> VPC B	-	pcx-01794b7f3a49a43a8
ewb-lab	EC2 key pair (RSA, .pem)	-	kept on my laptop only

Why these CIDRs? 10.10.0.0/16 and 10.20.0.0/16 do not overlap, and each /24 subnet sits inside its own VPC range. Peering cannot be created between VPCs with overlapping CIDRs, and different subnet ranges alone would not fix that.

04 / BUILD THE NETWORK

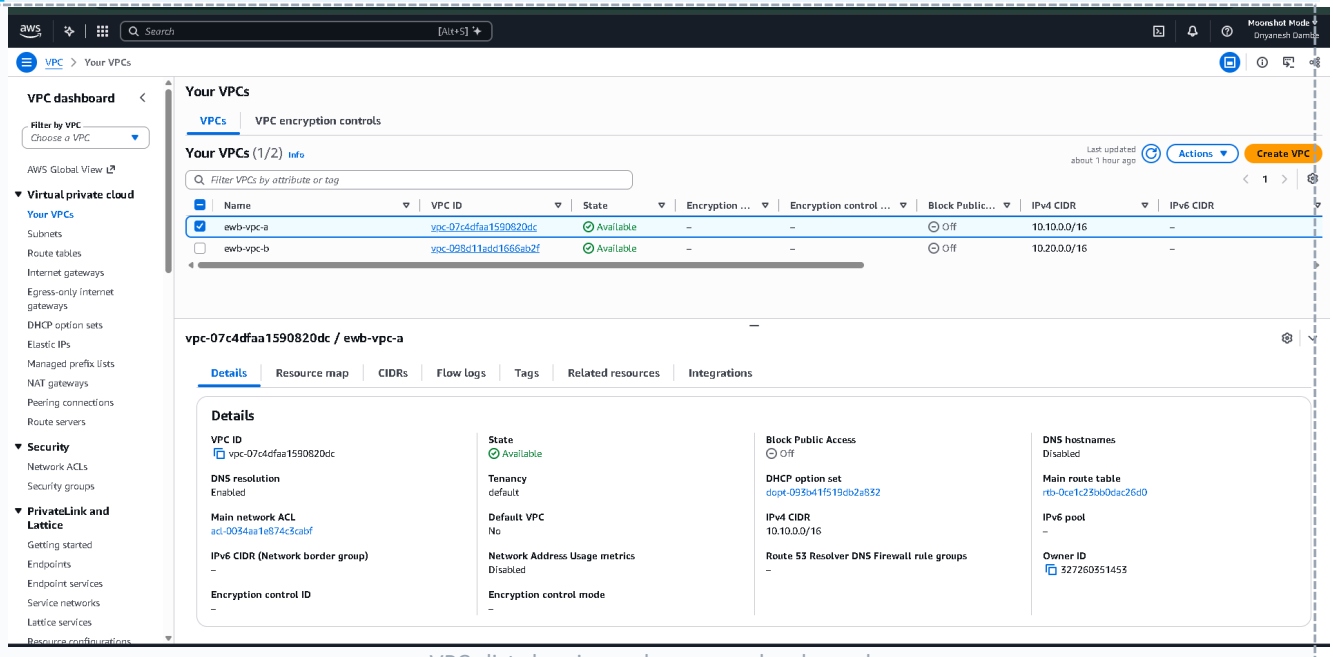
Create VPC A

VPC only, so I controlled every subnet and route.

- Created ewb-vpc-a (10.10.0.0/16) using the VPC only option, no IPv6, default tenancy.
- Created ewb-a-public (10.10.1.0/24) and ewb-a-private (10.10.2.0/24) in the same Availability Zone.
- Created ewb-igw-a and attached it to ewb-vpc-a.
- Created ewb-rt-a-public, associated it with ewb-a-public and added 0.0.0.0/0 to ewb-igw-a.
- Created ewb-rt-a-private, associated it with ewb-a-private and added no internet route.

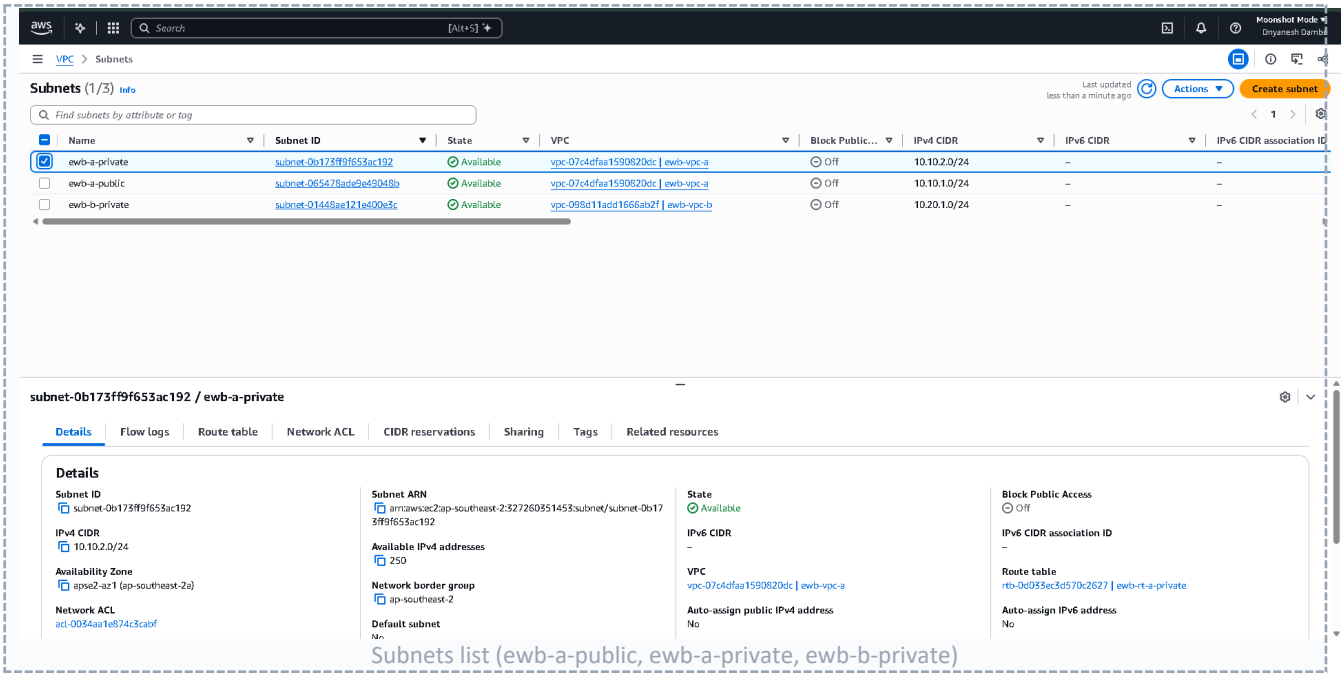
Checkpoint 01

Subnet associations and the public default route. The private route table contains only its local VPC route.



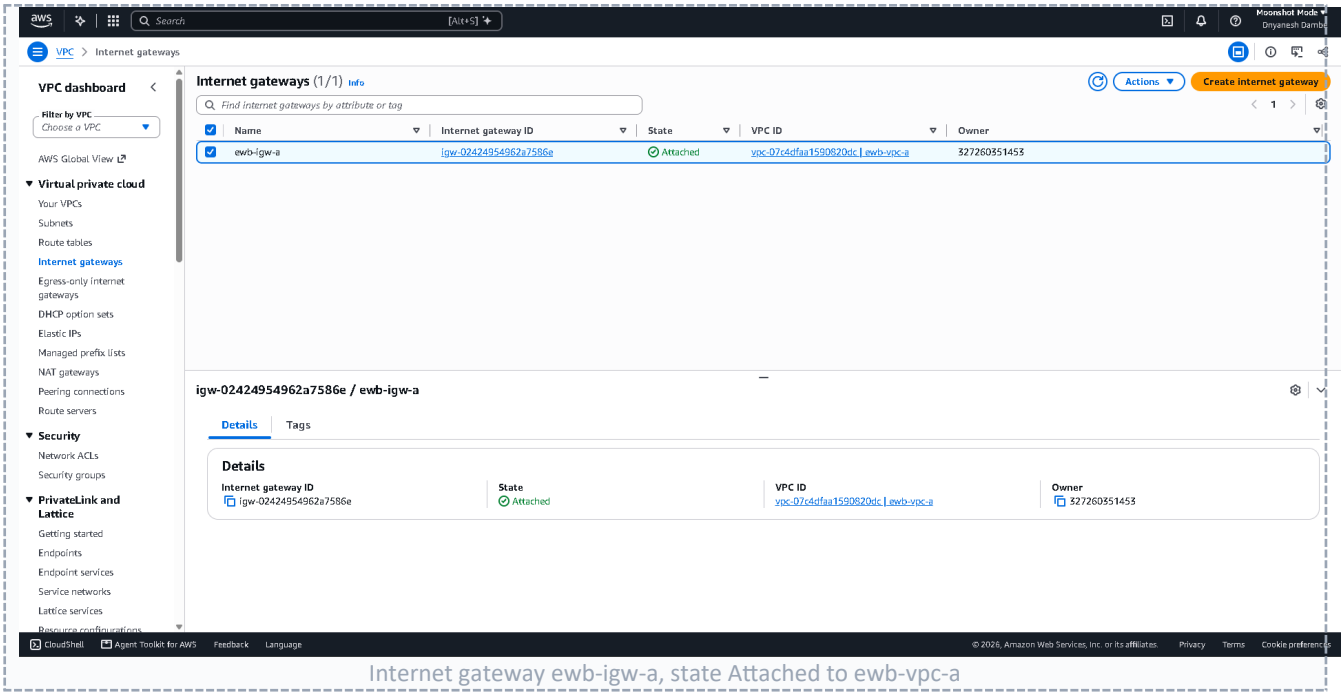
VPCs list showing ewb-vpc-a and ewb-vpc-b

Host: AWS Console **Test:** VPCs created **Result:** Both VPCs present with correct CIDRs



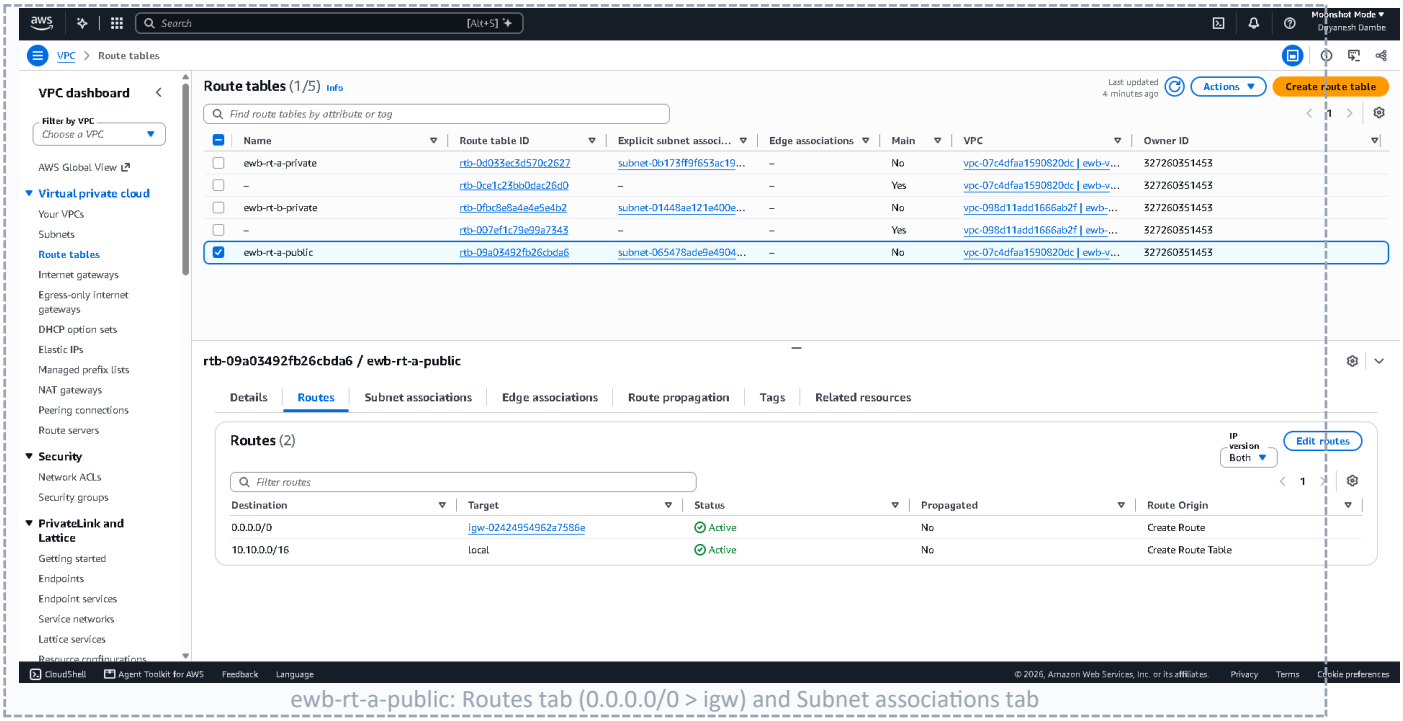
Subnets list (ewb-a-public, ewb-a-private, ewb-b-private)

Host: AWS Console **Test:** Subnets created **Result:** Three subnets with correct CIDRs



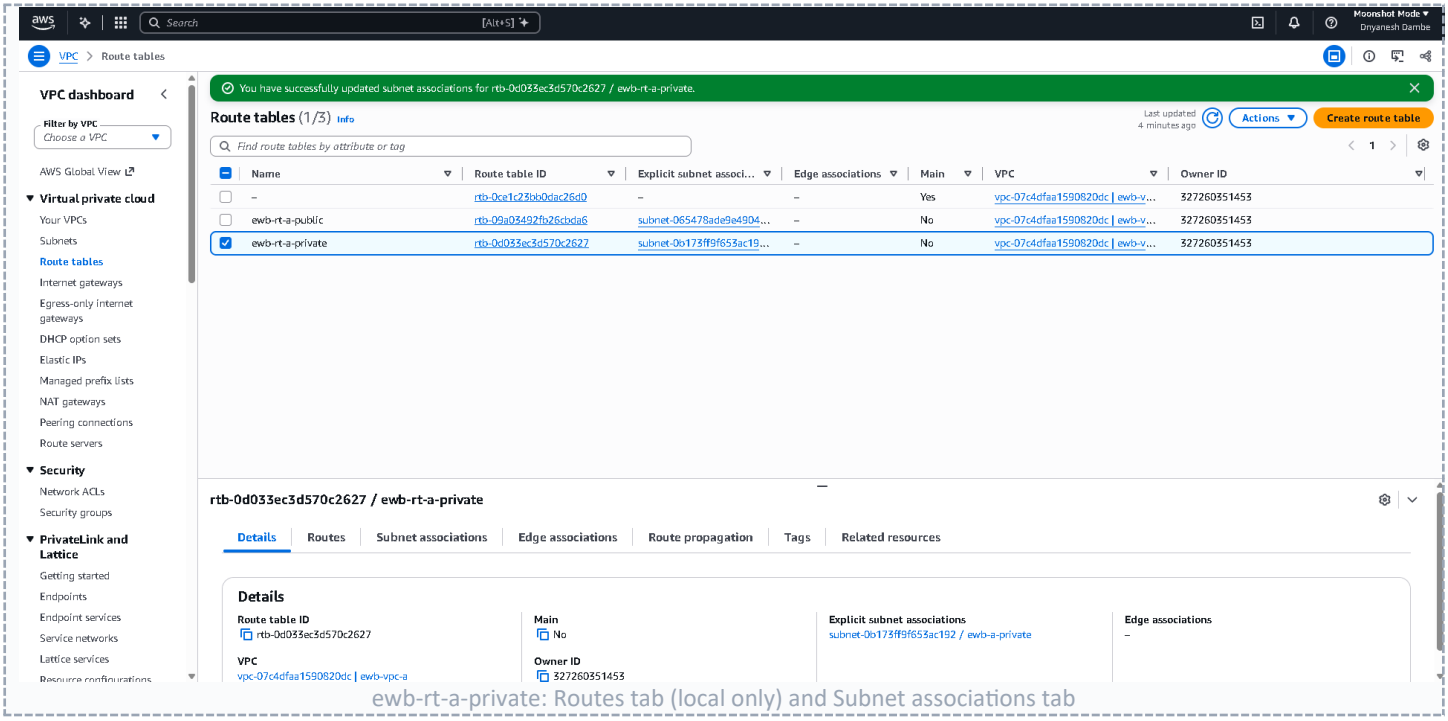
Internet gateway ewb-igw-a, state Attached to ewb-vpc-a

Host: AWS Console **Test:** IGW attached **Result:** Attached to VPC A only



ewb-rt-a-public: Routes tab (0.0.0.0/0 > igw) and Subnet associations tab

Host: AWS Console **Test:** Checkpoint 01 **Result:** Public route to IGW, ewb-a-public associated



ewb-rt-a-private: Routes tab (local only) and Subnet associations tab

Host: AWS Console **Test:** Checkpoint 01 **Result:** Local route only, ewb-a-private associated

Build VPC B. Set the rules.

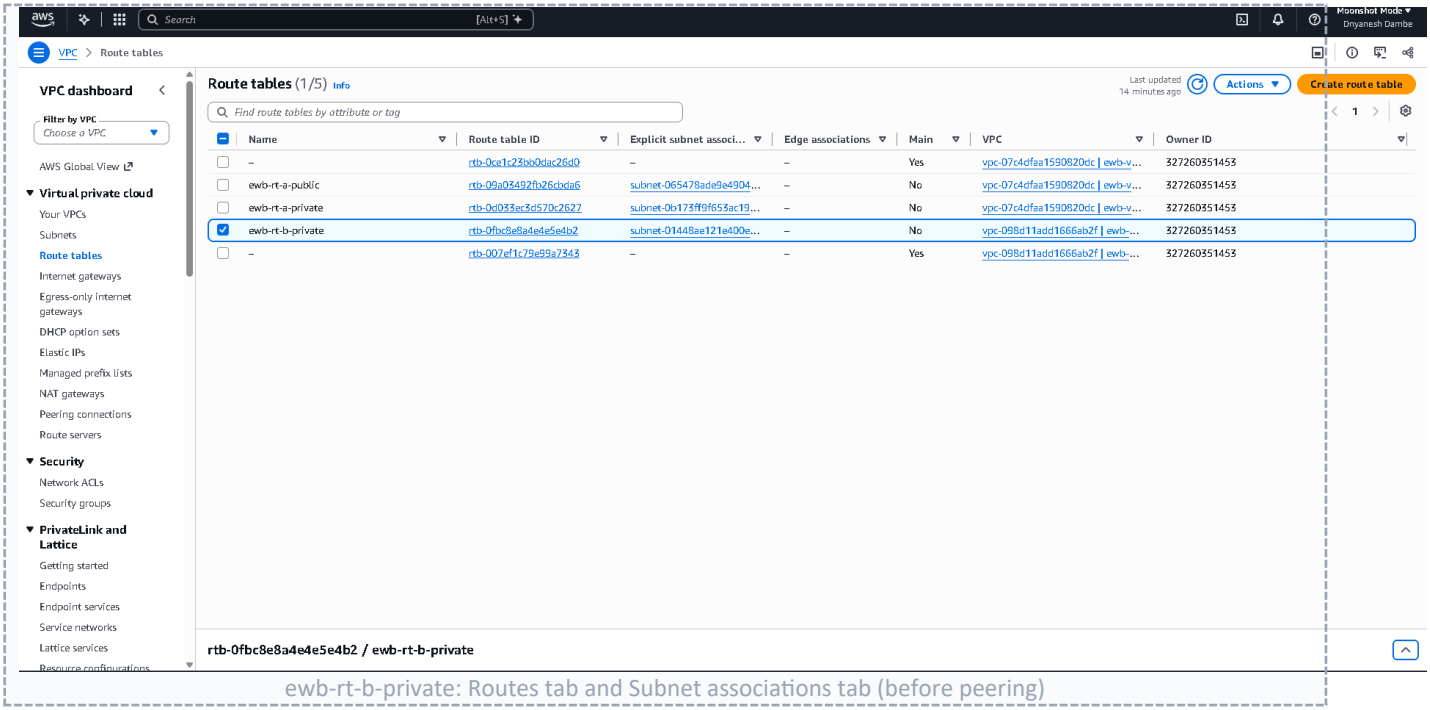
Both application servers stay private and inbound access is scoped to the lab hosts.

I created ewb-vpc-b (10.20.0.0/16), the subnet ewb-b-private (10.20.1.0/24) and the route table ewb-rt-b-private, explicitly associated with that subnet. VPC B has no internet gateway, so it is fully private.

Inbound security group rules

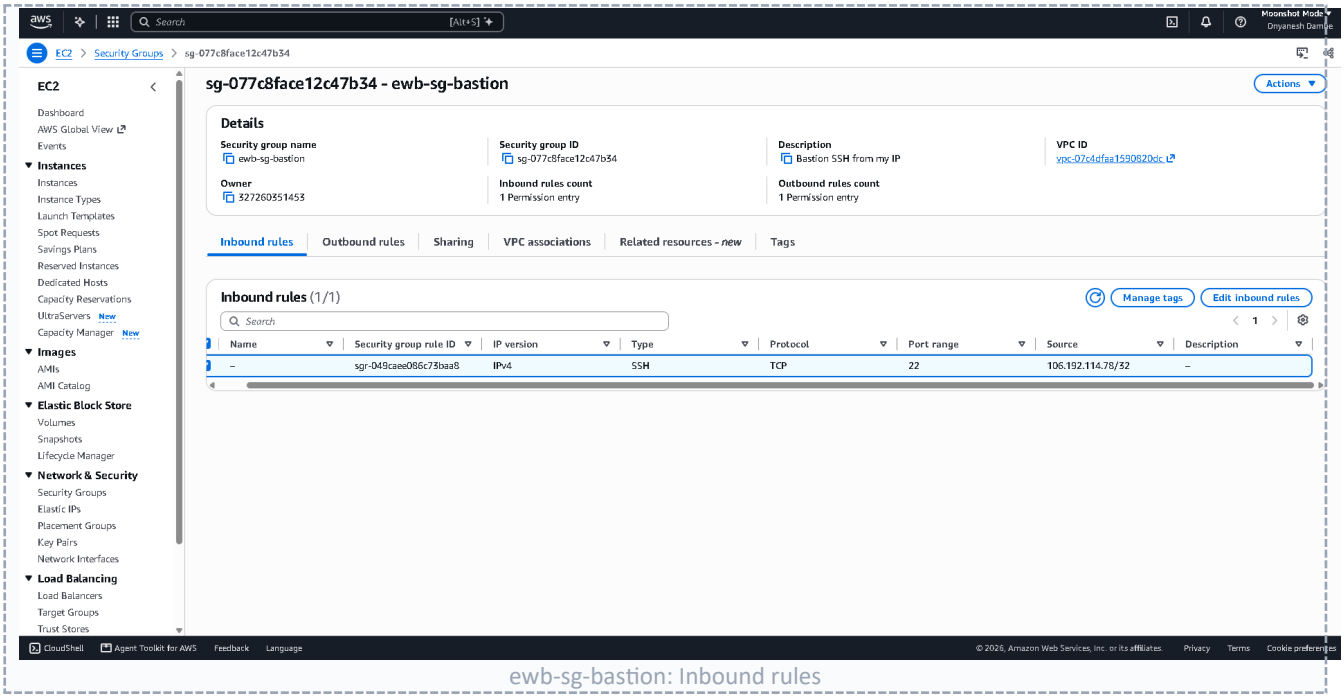
Security group	Type / port	Source
ewb-sg-bastion VPC A	SSH / TCP 22	My public IPv4 /32 106.192.114.78
ewb-sg-app-a VPC A	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a VPC A	Custom ICMP IPv4, Echo request	10.20.1.10/32 (App-B)
ewb-sg-app-b VPC B	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b VPC B	Custom ICMP IPv4, Echo request	10.10.2.10/32 (App-A)

Outbound rules were left at the default allow-all. Security groups are stateful, so replies to permitted traffic are allowed automatically. SSH is never opened to 0.0.0.0/0. The App-B SSH source is the bastion private IP because the bastion opens the connection to App-B.



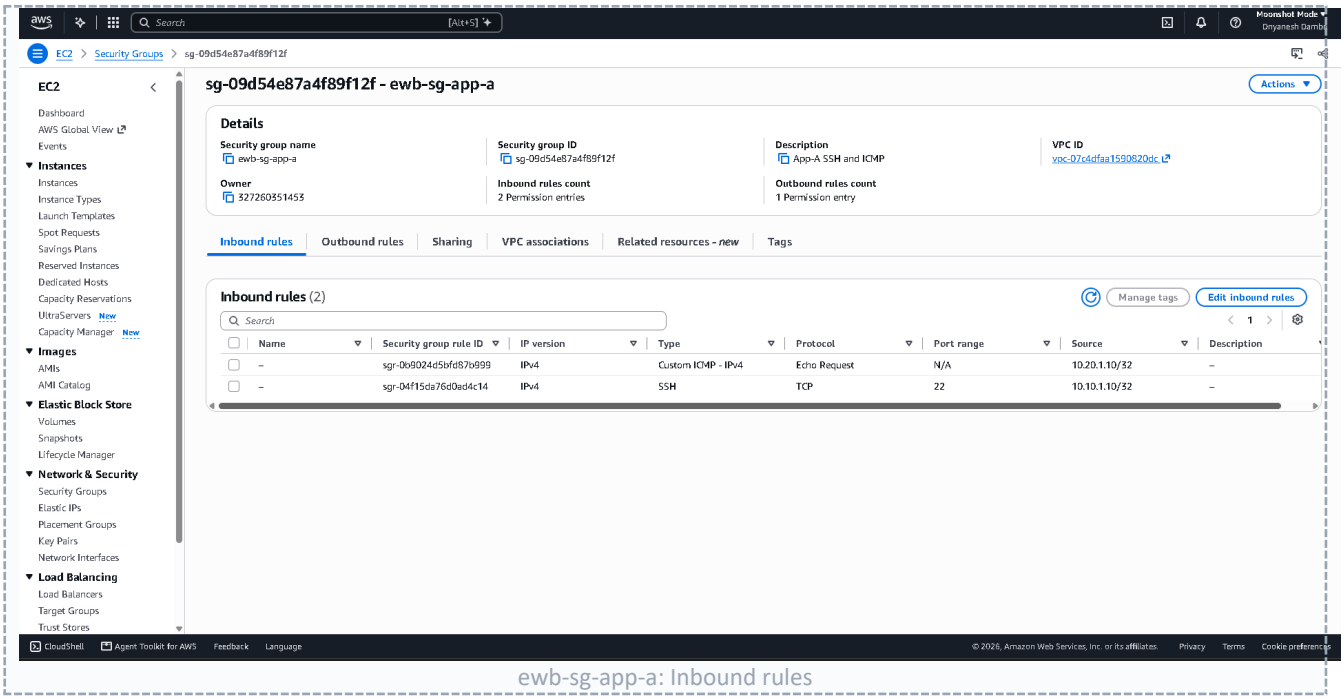
ewb-rt-b-private: Routes tab and Subnet associations tab (before peering)

Host: AWS Console **Test:** VPC B route table **Result:** Local route only, ewb-b-private associated



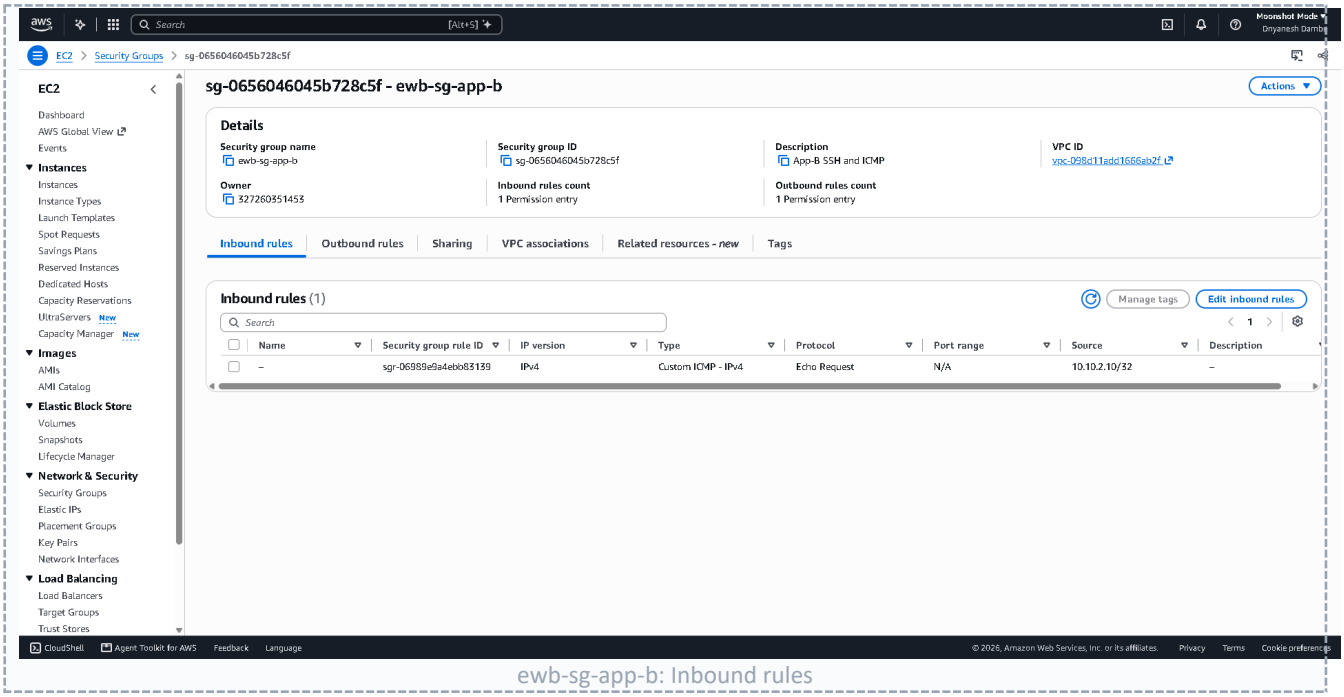
ewb-sg-bastion: Inbound rules

Host: AWS Console **Test:** Security group **Result:** SSH 22 from my IP /32



ewb-sg-app-a: Inbound rules

Host: AWS Console **Test:** Security group **Result:** SSH from bastion, ICMP from App-B



ewb-sg-app-b: Inbound rules

Host: AWS Console **Test:** Security group **Result:** SSH from bastion, ICMP from App-A

06 / COMPUTE

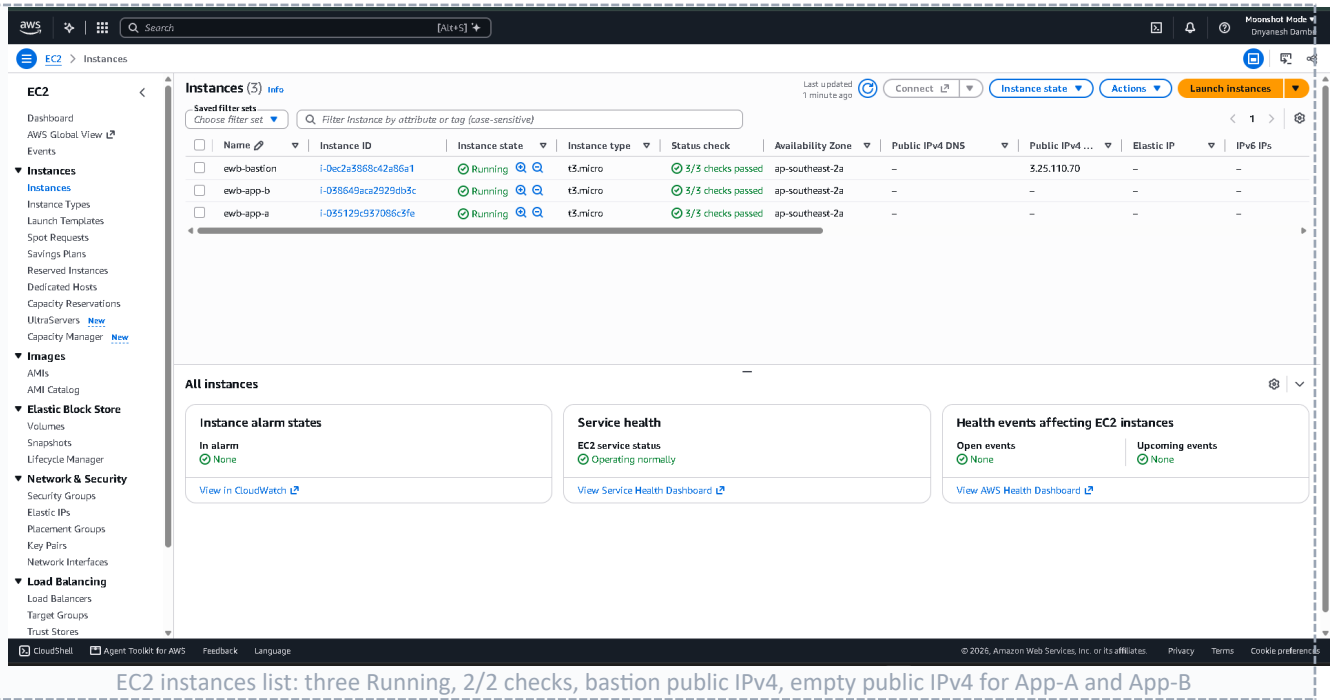
Launch the three instances

Same key pair and Amazon Linux username throughout.

I created an RSA key pair named ewb-lab, downloaded the .pem file and kept it only on my laptop. I launched three Amazon Linux 2023 t3.micro instances and set each primary private IP manually under Advanced network settings.

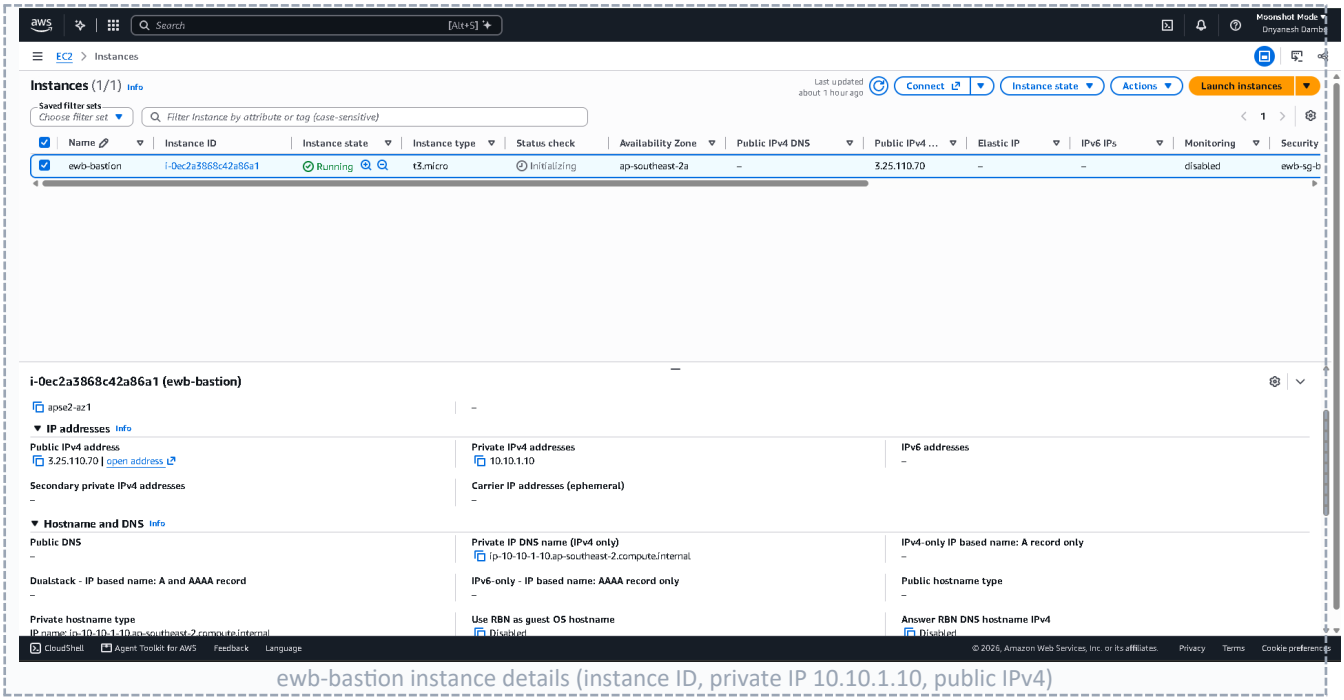
Instance	Network settings	Security group
ewb-bastion	VPC A / ewb-a-public 10.10.1.10 Auto-assign public IP: Enable	ewb-sg-bastion
ewb-app-a	VPC A / ewb-a-private 10.10.2.10 Auto-assign public IP: Disable	ewb-sg-app-a
ewb-app-b	VPC B / ewb-b-private 10.20.1.10 Auto-assign public IP: Disable	ewb-sg-app-b

Checkpoint 02
All three instance IDs and private IPs, the bastion public IPv4, and an empty Public IPv4 field for App-A and App-B.



EC2 instances list: three Running, 2/2 checks, bastion public IPv4, empty public IPv4 for App-A and App-B

Host: AWS Console **Test:** Checkpoint 02 / Isolation **Result:** No public IPv4 on App-A and App-B



ewb-bastion instance details (instance ID, private IP 10.10.1.10, public IPv4)

Host: AWS Console **Test:** Checkpoint 02 **Result:** Bastion has public IP

07 / ADMINISTRATION

Connect without copying keys

All SSH commands run on my laptop. ProxyJump uses the bastion as the connection path.

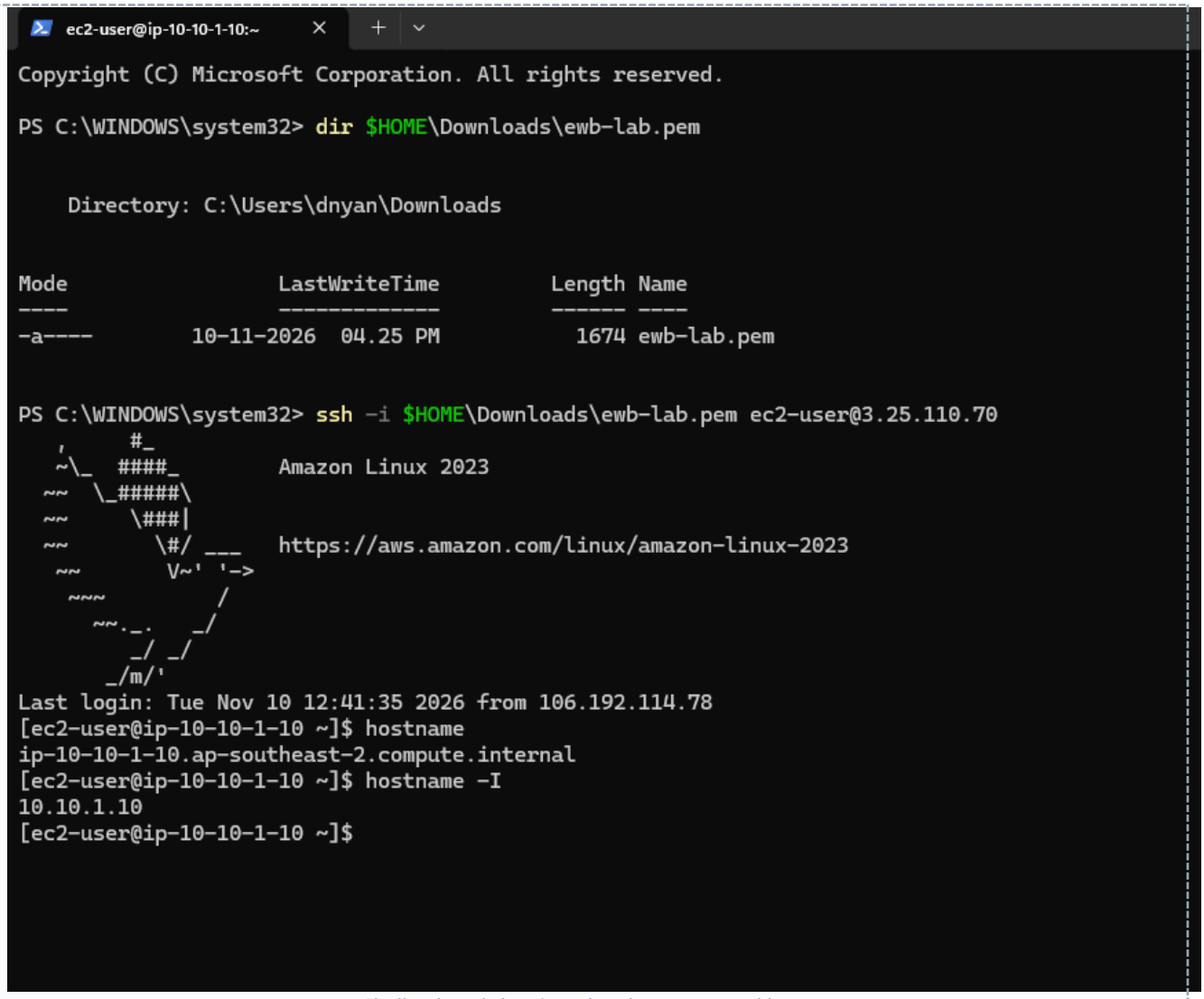
My laptop runs Windows, so I used the built-in OpenSSH client in PowerShell. The key stays in my Downloads folder and the config file is at C:\Users\dnyan\.ssh\config (saved without a .txt extension).

```
Host ewb-bastion
    HostName 3.25.110.70
    User ec2-user
    IdentityFile ~/Downloads/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/Downloads/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/Downloads/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

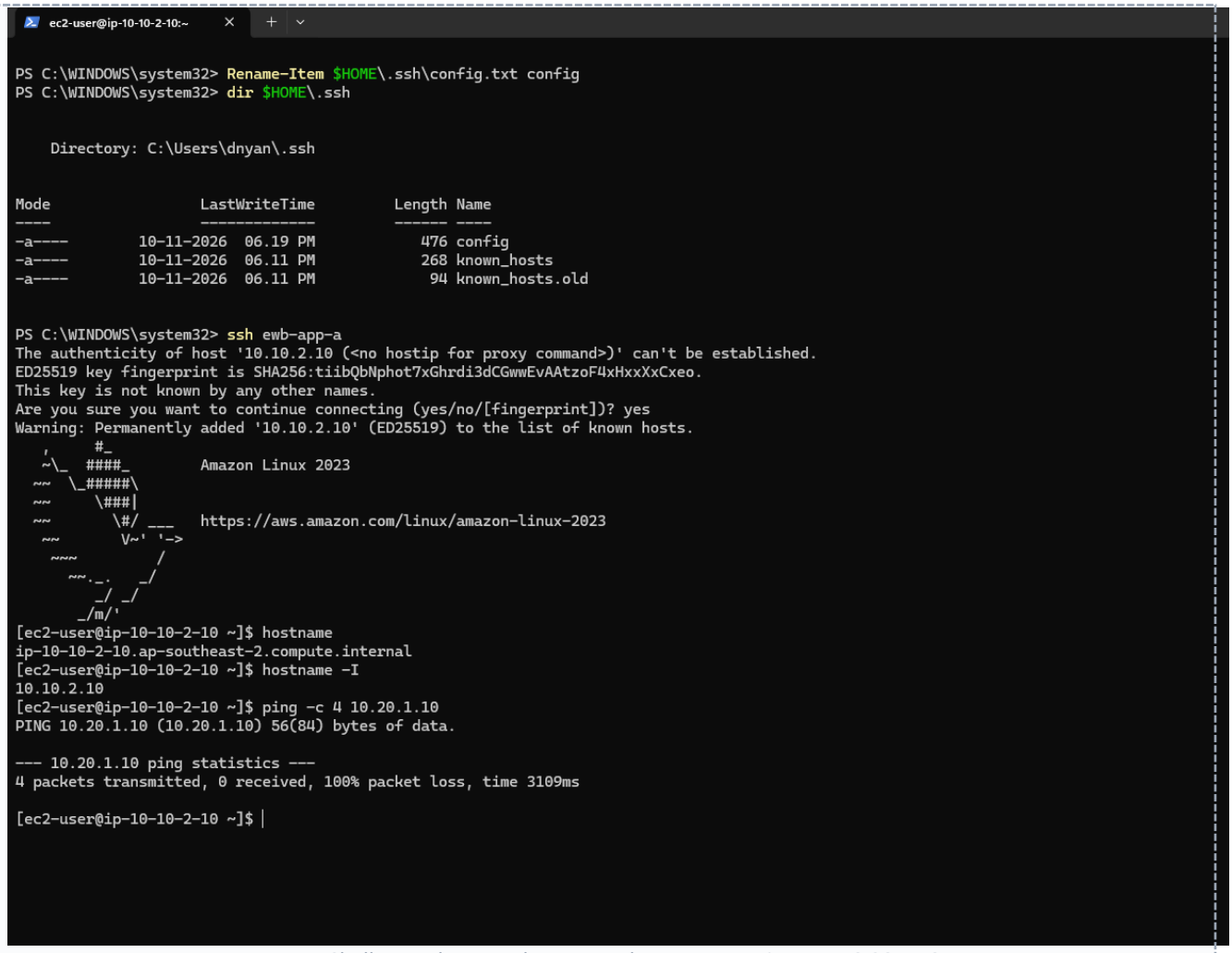
ssh ewb-bastion and ssh ewb-app-a both worked before peering because they use VPC A's local route. The private key was never copied onto any instance.



PowerShell: ssh ewb-bastion, then hostname and hostname -I

Host: ewb-bastion (10.10.1.10) **Test:** SSH to bastion **Result:** Connected; hostname -I shows 10.10.1.10

Test 01: Baseline before peering



PowerShell on ewb-app-a: hostname, hostname -I, ping -c 4 10.20.1.10

Host: ewb-app-a (10.10.2.10) **Test:** 01 Baseline **Result:** 4 packets transmitted, 0 received, 100% packet loss

This failure is expected. No peering connection or peer routes existed yet, so App-A had no path to VPC B. This is my "before" evidence.

08 / ESTABLISH PEERING

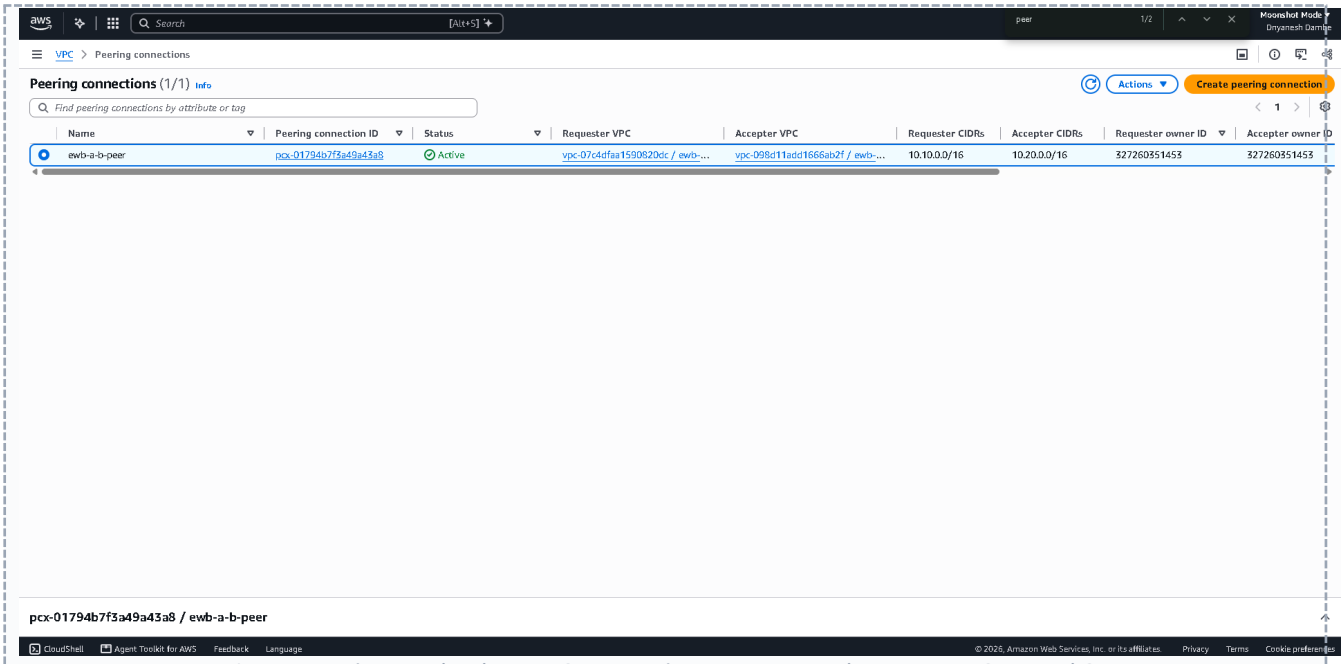
Request. Accept. Activate.

The peering connection is the link. Route tables and security groups decide whether traffic can use it.

Field	Value
Name	ewb-a-b-peer
VPC ID (Requester)	ewb-vpc-a [vpc-07c4dfaa1590820dc]
Account	My account
Region	This Region (ap-southeast-2)
VPC ID (Accepter)	ewb-vpc-b [vpc-098d11add1666ab2f]
Peering ID	pcx-01794b7f3a49a43a8

I created the request from VPC A, then accepted it with Actions > Accept request. Acceptance is required even when both VPCs are in the same account. The status changed from Pending acceptance to Active.

> **Checkpoint 03**
> Connection name, Active status and both VPC IDs. Active peering alone does not make ping or SSH work; routes are needed.



Peering connections: ewb-a-b-peer, Status Active, requester and accepter VPC IDs and CIDRs

Host: AWS Console **Test:** Checkpoint 03 **Result:** Peering Active

09 / ROUTING

Add routes on both sides

Each named route table, its subnet association and the routes I added.

ewb-rt-a-private / associated to ewb-a-private

Destination	Target	Purpose
10.10.0.0/16	local	Existing VPC A route
10.20.0.0/16	pcx-01794b7f3a49a43a8	App-A to VPC B

ewb-rt-b-private / associated to ewb-b-private

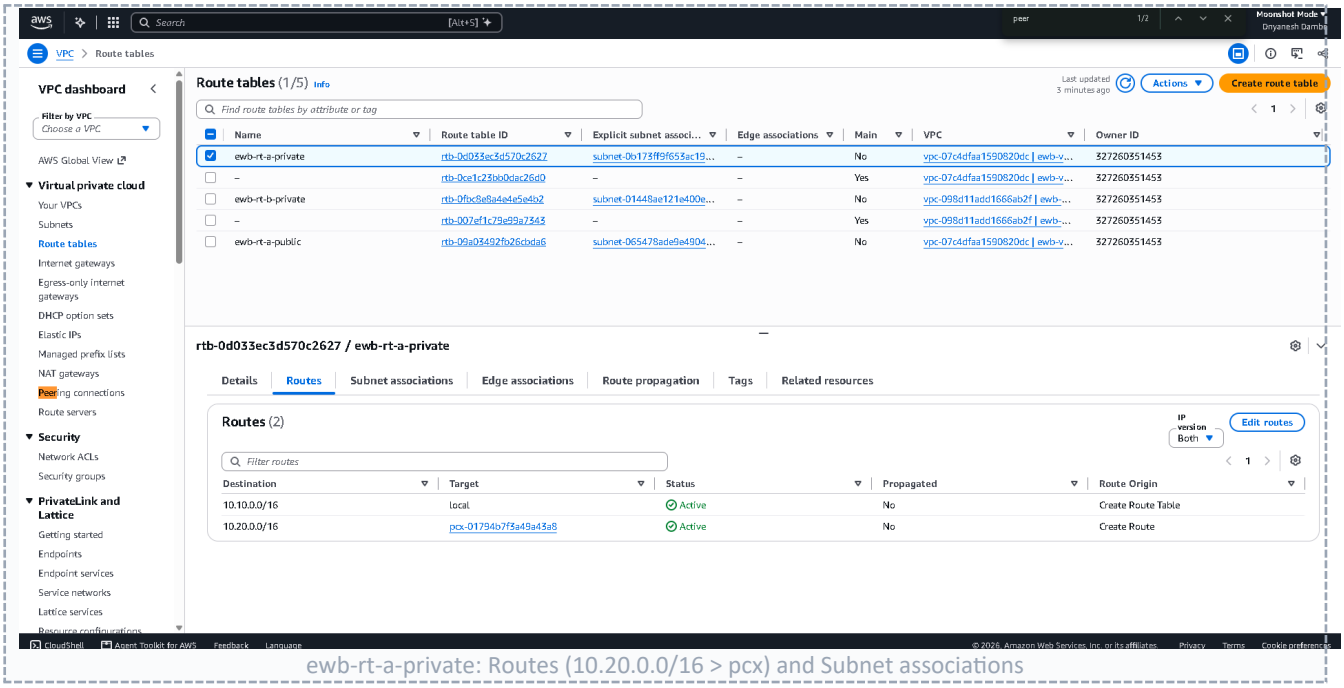
Destination	Target	Purpose
10.20.0.0/16	local	Existing VPC B route
10.10.0.0/16	pcx-01794b7f3a49a43a8	Return path to VPC A

ewb-rt-a-public / associated to ewb-a-public

Destination	Target	Purpose
10.10.0.0/16	local	Bastion to App-A
0.0.0.0/0	igw-02424954962a7586e	Bastion internet access
10.20.0.0/16	pcx-01794b7f3a49a43a8	Bastion SSH to App-B

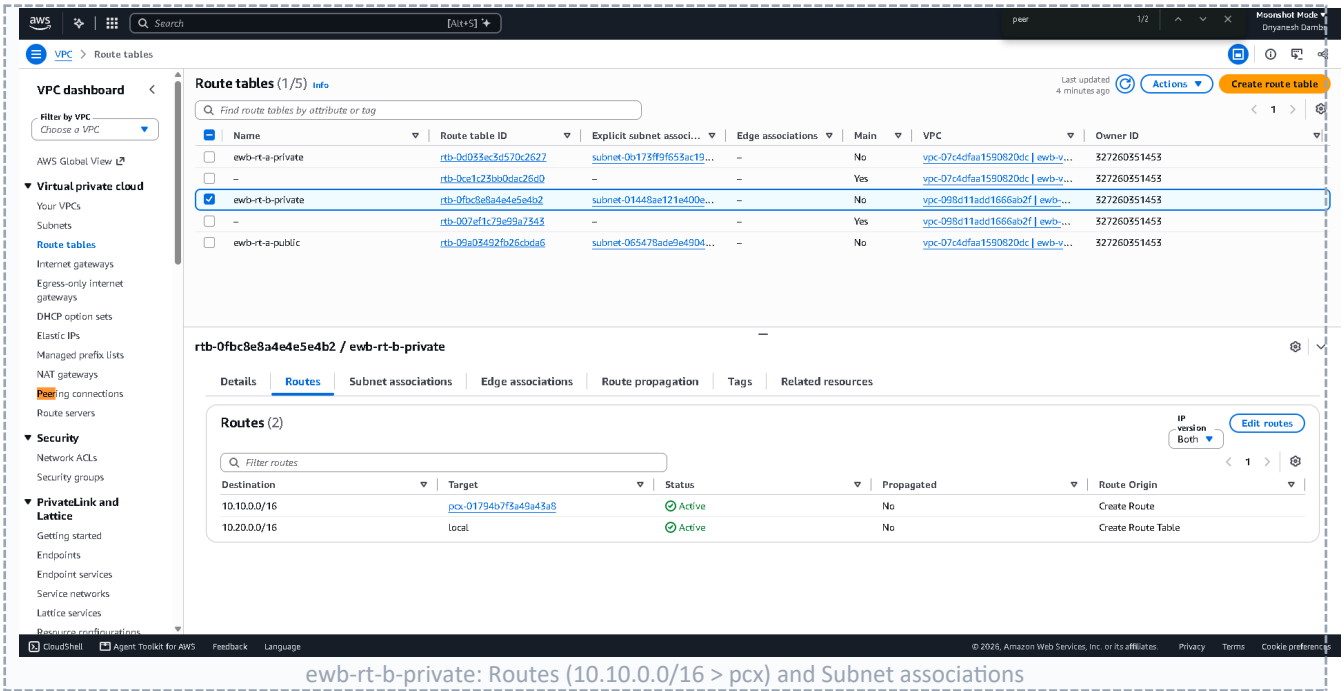
> **Why does the public subnet need a peering route?**
> ProxyJump makes the bastion open its own connection to App-B, so the bastion's subnet must know how to reach VPC B. The more specific 10.20.0.0/16 route wins over the internet default route.

Checkpoint 04: Routes and Subnet associations of all three tables are shown below. A one-way route is not enough, because replies also need a route back.



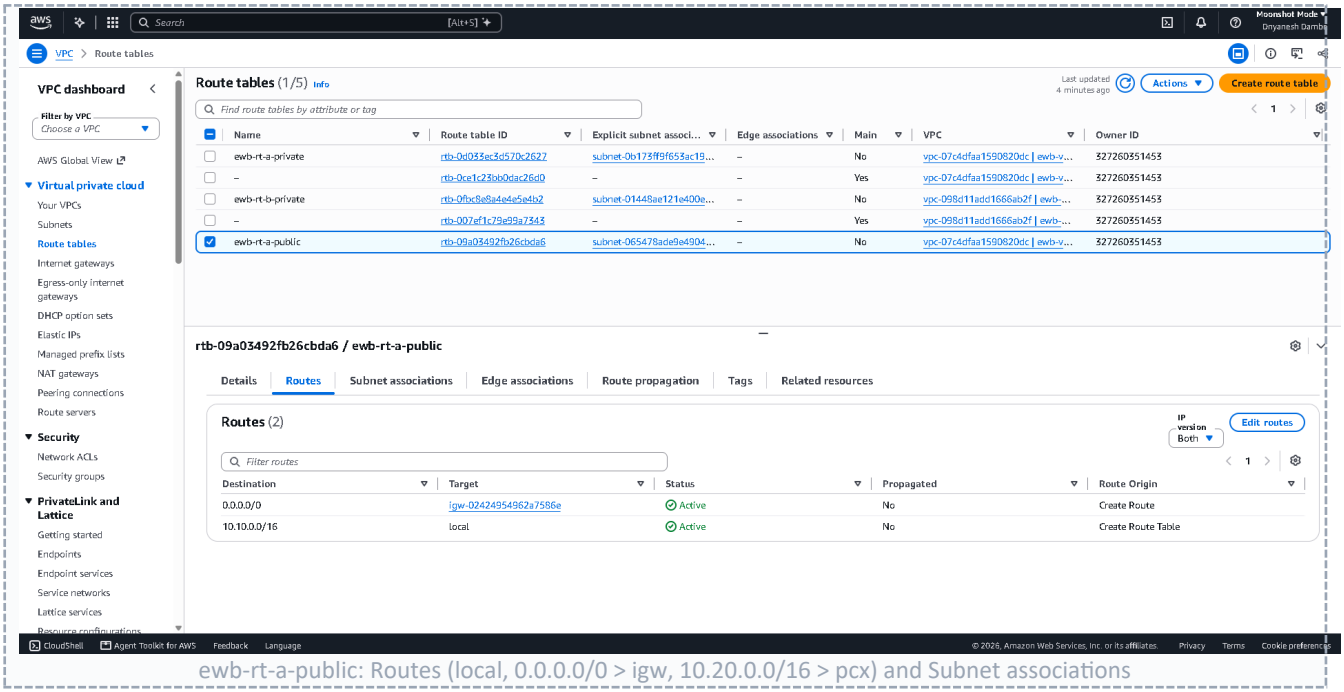
ewb-rt-a-private: Routes (10.20.0.0/16 > pcx) and Subnet associations

Host: AWS Console **Test:** Checkpoint 04 **Result:** Route Active



ewb-rt-b-private: Routes (10.10.0.0/16 > pcx) and Subnet associations

Host: AWS Console **Test:** Checkpoint 04 **Result:** Return route Active



ewb-rt-a-public: Routes (local, 0.0.0.0/0 > igw, 10.20.0.0/16 > pcx) and Subnet associations

Host: AWS Console **Test:** Checkpoint 04 **Result:** Peering route present alongside IGW route

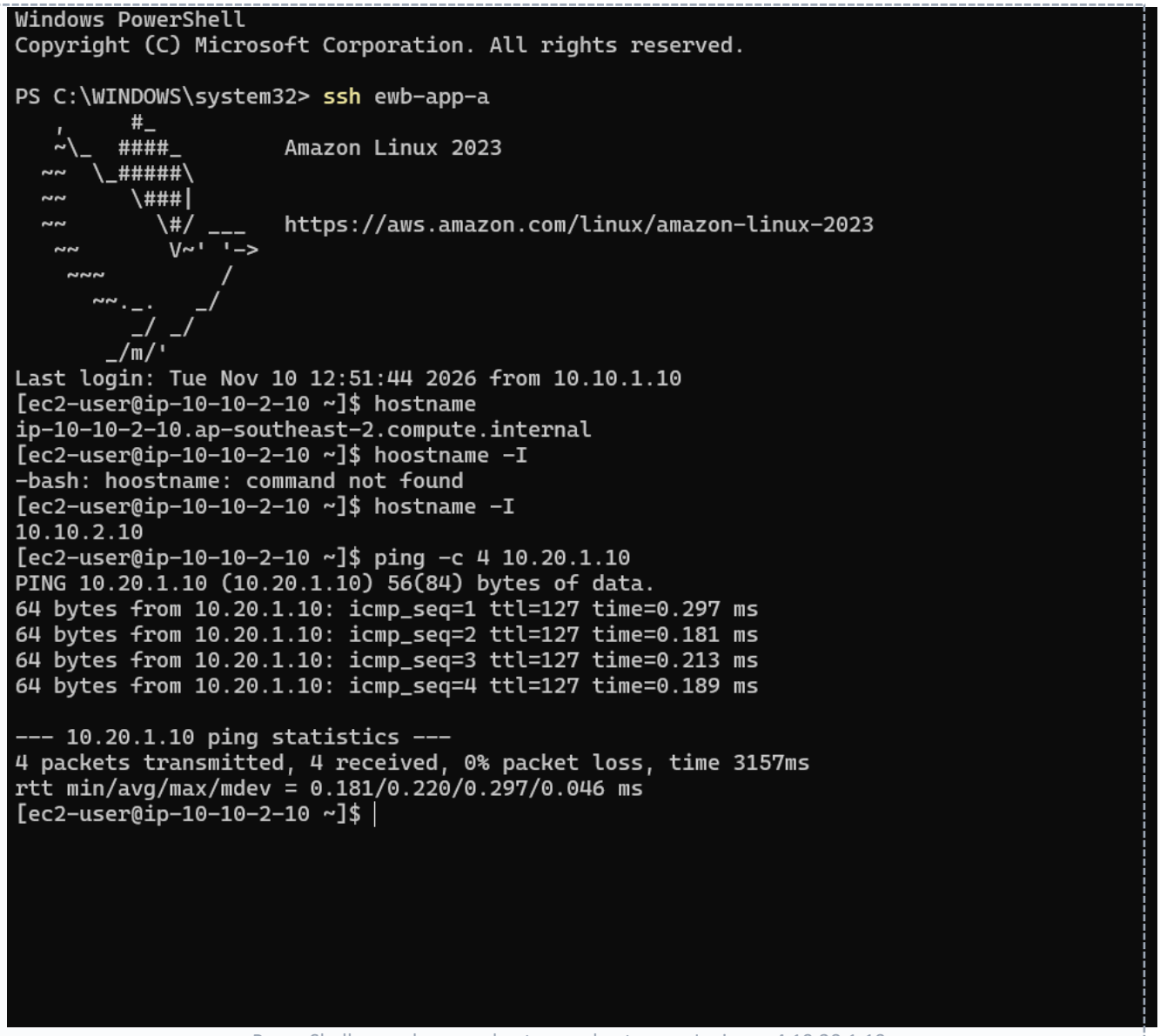
10 / VERIFICATION

Prove the project works

Real terminal output from my environment.

Test	Where / command	Expected	My result
01 Baseline	App-A before peering ping -c 4 10.20.1.10	No replies	100% packet loss (before)
02 Forward path	App-A after routes ping -c 4 10.20.1.10	4 replies, 0% loss	0% packet loss (4 of 4 received)
03 Reverse path	App-B after routes ping -c 4 10.10.2.10	4 replies, 0% loss	0% packet loss (4 of 4 received)
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B via bastion	Connected, prompt ip-10-20-1-10
05 Isolation	EC2 instance details	No public IPv4 on App-A / App-B	Confirmed

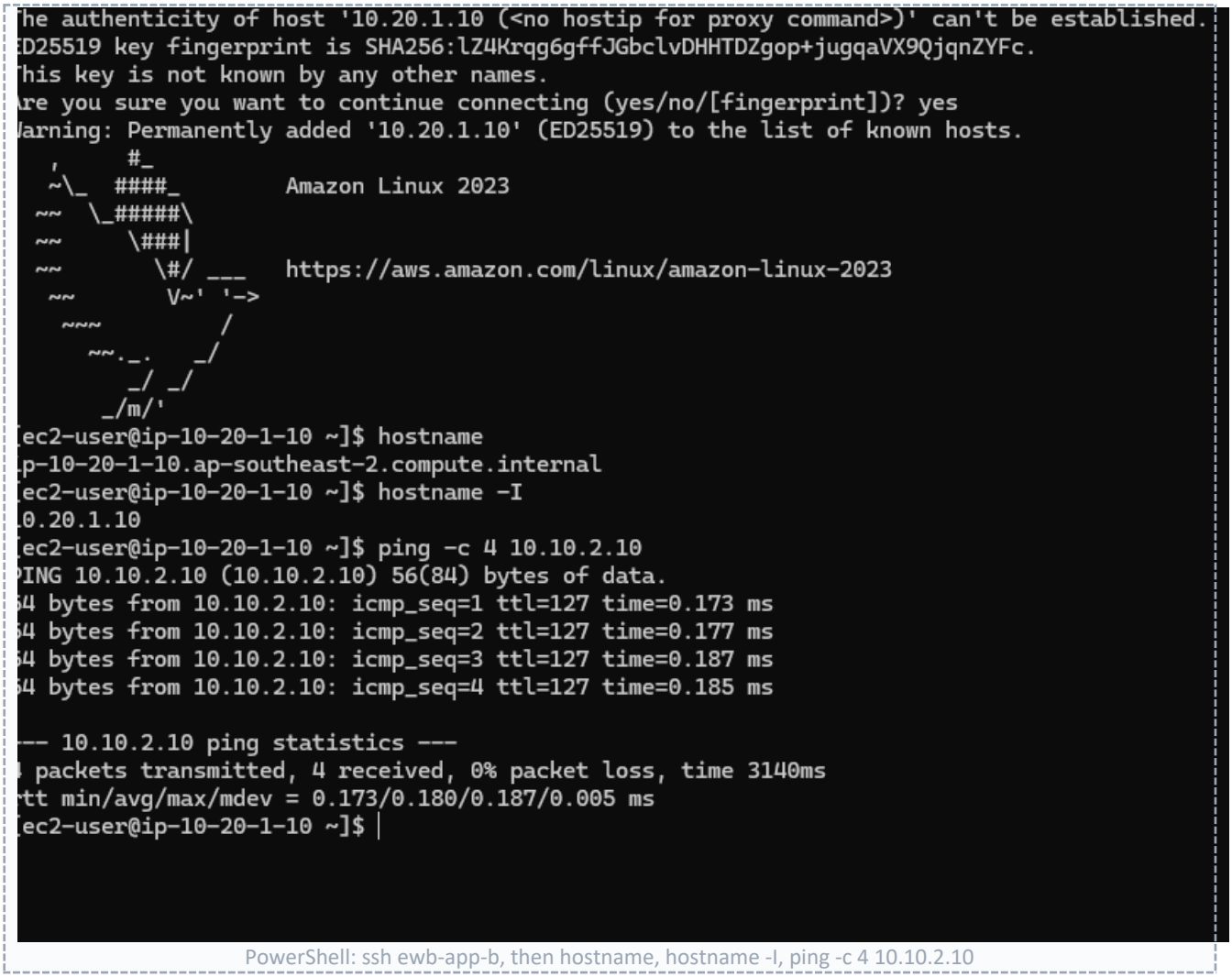
Test 02: Forward path



PowerShell on ewb-app-a: hostname, hostname -I, ping -c 4 10.20.1.10

Host: ewb-app-a (10.10.2.10) **Test:** 02 Forward path **Result:** 4 transmitted, 4 received, 0% packet loss

Tests 03 and 04: Private SSH and reverse path



PowerShell: ssh ewb-app-b, then hostname, hostname -I, ping -c 4 10.10.2.10

Host: ewb-app-b (10.20.1.10) **Test:** 03 Reverse / 04 Private SSH **Result:** Shell reached; 4 received, 0% packet loss

Test 05: Isolation

Both App-A and App-B show an empty Public IPv4 address field in the EC2 console (see the instances list in section 06). They are reachable only through the bastion and the private peering path.

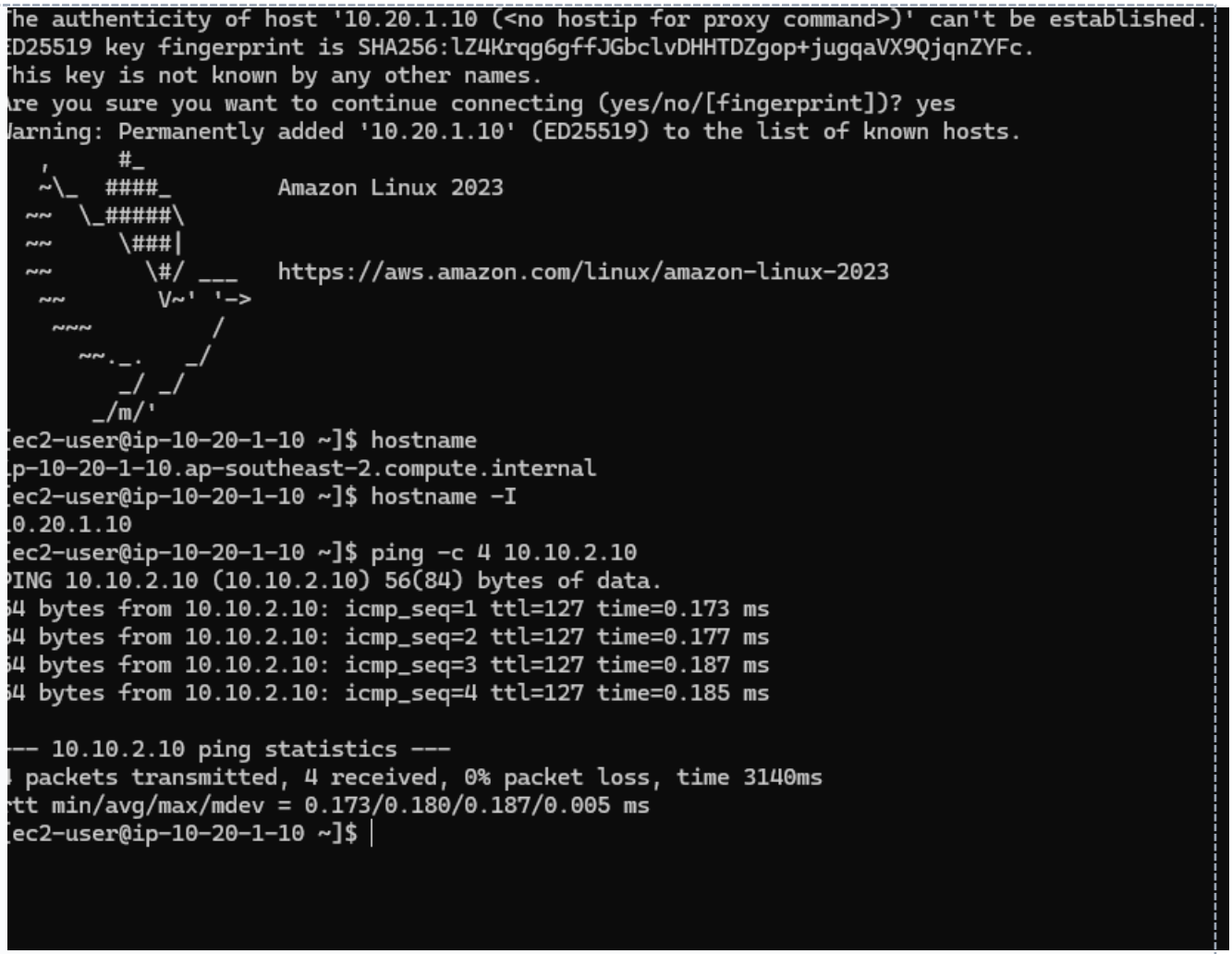
10 / VERIFICATION

Controlled failure and recovery

Proving the security group rule really controls the traffic.

In ewb-sg-app-b I deleted only the inbound ICMP Echo request rule from 10.10.2.10/32 and kept the SSH rule. From App-A the ping to 10.20.1.10 then failed with 100% packet loss. I re-added the exact same rule (Custom ICMP IPv4, Echo request, source 10.10.2.10/32) and the ping returned to 0% loss.

This shows that routes and active peering are not enough on their own. The destination security group must also allow the traffic, and the rule has to sit on the destination side.



ewb-sg-app-b inbound rules BEFORE removal (SSH and ICMP)

Host: AWS Console **Test:** Controlled failure: before **Result:** Both rules present



App-A: ping -c 4 10.20.1.10 after ICMP rule removed

Host: ewb-app-a (10.10.2.10) **Test:** Controlled failure **Result:** 100% packet loss



```
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Tue Nov 10 13:11:27 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ hostname
ip-10-10-2-10.ap-southeast-2.compute.internal
[ec2-user@ip-10-10-2-10 ~]$ hostname -I
10.10.2.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.182 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.175 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.200 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.177 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3120ms
rtt min/avg/max/mdev = 0.175/0.183/0.200/0.009 ms
[ec2-user@ip-10-10-2-10 ~]$
[ec2-user@ip-10-10-2-10 ~]$
[ec2-user@ip-10-10-2-10 ~]$
[ec2-user@ip-10-10-2-10 ~]$ #    controlled failure
[ec2-user@ip-10-10-2-10 ~]$
[ec2-user@ip-10-10-2-10 ~]$
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3141ms

[ec2-user@ip-10-10-2-10 ~]$ client_loop: send disconnect: Connection reset
client_loop: send disconnect: Unknown error
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> ssh ewb-app-a
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Tue Nov 10 13:16:46 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.155 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.196 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.160 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.174 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3133ms
rtt min/avg/max/mdev = 0.155/0.171/0.196/0.015 ms
[ec2-user@ip-10-10-2-10 ~]$
```

App-A: ping -c 4 10.20.1.10 after rule restored

Host: ewb-app-a (10.10.2.10) **Test:** Recovery **Result:** 0% packet loss

11 / EXPLANATION

Forward and return path

How a ping travels between the two private servers.

Forward path (App-A to App-B)

App-A (10.10.2.10) sends an ICMP echo request to 10.20.1.10. The route table ewb-rt-a-private matches 10.20.0.0/16 and sends the packet to the peering connection ewb-a-b-peer. It arrives in VPC B, where ewb-sg-app-b allows ICMP Echo request from 10.10.2.10/32, and App-B receives it.

Return path (App-B to App-A)

App-B replies to 10.10.2.10. The route table ewb-rt-b-private matches 10.10.0.0/16 and sends the reply back through the same peering connection to VPC A. The security group is stateful, so the reply is allowed without an extra inbound rule.

SSH path (laptop to App-B)

My laptop connects to the bastion's public IP through ewb-igw-a. From the bastion, SSH opens a new connection to App-B at 10.20.1.10. This needs the 10.20.0.0/16 > peering route in ewb-rt-a-public, the return route in ewb-rt-b-private, and an SSH rule on ewb-sg-app-b that allows the bastion's private IP 10.10.1.10/32.

Be ready to explain (viva)

1. Why is the return route needed?

A route only describes where packets can go in one direction. If VPC B has no route to 10.10.0.0/16, App-B receives the request but cannot send the reply back, so the ping looks like it failed.

2. Why is the App-B SSH source the bastion private IP?

With ProxyJump the bastion opens the connection to App-B, so App-B sees the bastion's private address 10.10.1.10 as the source, not my laptop's public IP.

3. Why does Active peering not guarantee a successful ping?

Peering is only the link. Traffic also needs routes in both VPCs, a security group rule on the destination, and (in custom setups) network ACLs that allow it. Peering is a link, not permission.

4. Can VPC B use VPC A's internet gateway over peering?

No. Peering is not transitive and does not forward traffic to a gateway in the other VPC, so VPC B stays without internet access.

12 / TROUBLESHOOTING

Problems I faced

What went wrong during the build and how I fixed it.

Problem	Cause	Fix
Bastion SSH timed out	My home public IP had changed, so the /32 rule in ewb-sg-bastion no longer matched.	Set the SSH source to My IP again and saved the rule.
SSH config not recognised	Notepad saved the file as config.txt.	Renamed it with Rename-Item to config and verified with dir.
App-B unreachable before peering	No peering connection or routes existed yet.	Expected result; fixed by creating peering and the three routes.
Key path depended on my Windows username	The IdentityFile line used a full C:/Users/... path, which I wanted to avoid.	Used ~/Downloads/ewb-lab.pem in the SSH config so the path works under any username.

Region note
My resources are in Asia Pacific (Sydney), ap-southeast-2. The workbook uses Mumbai only as an example; the design is identical in any single Region.

13 / REFLECTION

What I learned

A short note in my own words.

A VPC is my own private network in AWS, a subnet is a smaller slice of it, a route table decides where traffic from a subnet can go, and an internet gateway is the door to the internet. A subnet is only public because its route table points 0.0.0.0/0 to the internet gateway, and my private subnets never had that route. Peering taught me that the connection alone does nothing: I needed a route in each VPC, one for the request and one for the reply, and a security group rule on the destination. The controlled failure made this clear, because deleting one ICMP rule stopped the ping even though peering and routes were fine. The bastion is the only machine with a public IP, so the two app servers stay hidden and I reached them with ProxyJump without copying my key anywhere. I also hit two small problems, a changed home IP that blocked SSH and a config file saved as config.txt, and fixing them showed me how to read the error and check the security group first.

> **Declaration**
> I built this project myself in my own AWS account. The IDs, terminal output and screenshots in this report come from my own execution. No private key content is included.

Name: Dnyanesh Dattatray Dambe
Batch: AWS Batch 1
Date: 10 November 2026